CookCart — PRD & Implementation Plan

Recipe-to-Checkout Grocery Agent, powered by Prava Event: Prava Agentic Commerce Hackathon (48 hours) **Author:** Shreyas **Status:** Build-ready v2 **Scope lock:** Zepto + Swiggy Instamart only. Prava's general shopping skill (`prava-shopping`) is explicitly out of scope for this build. **Build agent:** Google Antigravity

---

Part A — Why this exists

A.1 The problem, in plain language

When someone decides to cook a specific dish, getting the ingredients requires them to personally do five separate jobs:

1. Work out exactly what the recipe needs, scaled correctly for how many people they're feeding
2. Remember what they already have in the kitchen, so they don't buy duplicates
3. Figure out which pack size to buy for each ingredient (a recipe needs 100ml of cream, the app only sells 200ml — now what?)
4. Check two or three different apps to see which one has the better price or actually has the item in stock
5. Build and pay for one or two separate carts

Nobody currently does all five of these for you. A recipe website gives you a list and stops — that's step 1 only. A quick-commerce app is a single-store cart — that's step 4 and 5 for one platform, not a comparison. There is a real, obvious gap between "I know what I want to cook" and "the ingredients are paid for and on the way," and today a human has to manually bridge that gap every single time they cook something.

A.2 Why this is worth building at this hackathon specifically

The hackathon brief says: *"Most AI products stop after giving an answer. We want to see what happens when the agent can finish the job."*

A recipe assistant that prints you a shopping list is the textbook example of an AI product that stops after giving an answer. CookCart's entire premise is refusing to stop there — the agent doesn't just tell you what to buy, it decides, compares, and completes the purchase, inside limits the user explicitly set.

It's also a strong hackathon build for practical reasons:

- The Zepto and Swiggy Instamart checkout mechanics already exist as ready-made Prava skills — you are not building payment plumbing from zero, you're building the decision layer on top of it
- It produces a genuinely visible, understandable live demo in under a minute

- It produces a genuinely visible, understandable live demo in under a minute
- It has real, nontrivial agent decisions in it (which pack size, which platform, what to skip) rather than being a single API call wearing an "AI" label

A.3 Honest framing

This is not a "someone is losing money right now" problem the way a missed subscription renewal is. It is a **friction and time problem** — a repeated, mildly annoying chore that a good agent can fully absorb. The pitch should lean on the completeness of the loop (discover → decide → transact, entirely unattended past the approval step) rather than oversell it as solving financial pain it doesn't solve.

Part B — Goal definition for the build agent

This section is written as a direct brief for the coding agent (Google Antigravity) executing this build. Read this before writing any code.

Primary goal: Build a working web application where a user types a dish and a serving count, and the system autonomously produces a completed, paid grocery order on Zepto or Swiggy Instamart — using real ingredient reasoning, a real pantry check, a real price/availability comparison between the two platforms, and a real Prava-authorized payment. No part of the checkout should be faked, hardcoded, or simulated for the demo. If a component cannot be made real in the time available, it must be explicitly cut from scope (see Part E) rather than mocked and presented as working.

Definition of done for the hackathon submission:

- A user can type "I'm making butter chicken for 4" into the app
- The system shows a structured ingredient list with quantities
- The system shows which items were skipped because they're pantry staples
- The system shows a real price/availability comparison between Zepto and Swiggy Instamart for the remaining items
- The system respects a user-configured Prava spend mandate and triggers Passkey approval correctly
- The system completes an actual checkout on the chosen platform via a real Prava one-time payment credential
- The user sees a clear final confirmation: what was bought, what was skipped, total cost, delivery ETA

Explicit non-goals for this build (do not spend time here): the general Prava shopping skill, more than two merchant platforms, a mobile app, auto-learned pantry detection, full nutrition tracking. See Part E for the full out-of-scope list.

Part C — Worked example (build and test against this exact case)

User input: *"I'm cooking butter chicken for 4 people this weekend."*

Step 1 — Recipe decomposition

An LLM call expands the dish + serving count into a structured list:

Ingredient	Quantity needed (serves 4)
Boneless chicken thighs	600 g
Yogurt (curd)	150 g
Butter	60 g
Heavy cream	100 ml
Tomato puree / tomatoes	400 g
Onion	2 medium
Ginger-garlic paste	2 tbsp
Garam masala	1 tbsp
Kashmiri red chili powder	1 tbsp
Turmeric	1 tsp
Cumin seeds	1 tsp
Kasuri methi (dried fenugreek)	1 tbsp
Salt	to taste

Step 2 — Pantry diff

The user's saved staples list marks salt, turmeric, and cumin seeds as "always have." These three are removed from the buy list. 13 items become 10.

Step 3 — Pack-size resolution

Every remaining ingredient gets mapped to the closest sensible real SKU on each platform, rounding up rather than under-buying, and the reasoning is shown to the user rather than hidden:

Ingredient	Needed	Zepto option	Swiggy Instamart option	Agent's pick

Chicken thighs	600 g	500g (₹220) / 1kg (₹410)	500g (₹235)	Zepto 500g — closest match, cheaper
Heavy cream	100 ml	200ml (₹65)	Out of stock	Zepto — only option in stock
Kasuri methi	1 tbsp	100g (₹45)	90g (₹42)	Swiggy — cheaper, shelf-stable so extra is fine
(remaining rows follow the same pattern)				

Step 4 — Cross-platform cart optimization

The agent builds full candidate carts, not just item-by-item comparisons, because delivery fees change the math:

- Cart A — everything on Zepto: ₹612 total, 1 delivery, ETA 14 min
- Cart B — everything on Swiggy Instamart: ₹649 total, 1 delivery, ETA 19 min
- Cart C — cheapest item split across both: ₹580 combined, but two delivery fees

Rule: only split across platforms if the item-level savings exceed the cost of the second delivery fee. In this case, splitting doesn't clear that bar, so the agent recommends Cart A.

Step 5 — Spend cap check + Prava mandate

The user has set: *"Zepto/Swiggy Instamart, up to ₹800 per grocery run, auto-approve under ₹500, confirm above ₹500."* ₹612 is under the ₹800 cap but above the ₹500 auto-approve line, so the agent surfaces a confirmation card (recipe, ingredient list, price, platform) and waits for the user to approve via Passkey. Once approved, Prava issues a one-time Zepto-scoped payment credential and the agent completes checkout with it.

Step 6 — Confirmation

"Ordered your butter chicken ingredients from Zepto — ₹612, arriving in ~14 minutes. I skipped salt, turmeric, and cumin since you already have those."

This exact sequence should be the primary rehearsed demo run.

Part D — Functional & non-functional requirements

D.1 Functional requirements

#	Requirement	Priority

F1	Parse a natural-language dish + serving count into a structured, quantity-scaled ingredient list	Must
F2	Store a per-user pantry/staples list and apply it to exclude already-owned items	Must
F3	Query the Zepto skill and the Swiggy Instamart skill for per-ingredient SKU availability and price	Must
F4	Resolve needed quantity to the nearest sensible pack size per platform, rounding up	Must
F5	Compare total cost (items + delivery) across both platforms and select the better single-platform cart	Must
F6	Enforce the user's Prava spend mandate before any checkout is attempted	Must
F7	Trigger Prava Passkey approval for any order above the auto-approve threshold	Must
F8	Complete checkout via a real Prava one-time payment credential on the chosen platform	Must
F9	Present a clear itemized confirmation: items bought, items skipped, total, ETA	Must
F10	Split the cart across both platforms when savings exceed the extra delivery fee	Should
F11	Deduplicate shared ingredients across a multi-dish weekly plan	Should
F12	Propose a substitute ingredient when something is out of stock on both platforms	Could

D.2 Non-functional requirements

- **Transparency is mandatory, not optional.** Every decision the agent makes (platform choice, pack-size rounding, skipped items) must be visible to the user, never silent. This is the actual trust mechanism of the whole product, not a UX nicety.
- **Hard spend enforcement.** The agent must never be able to exceed the Prava mandate cap through any code path — this should be enforced at the Prava layer as the real backstop, with application-level checks as a first line of defense, not the only line.
- **Demo-time latency.** Recipe input → checkout confirmation should complete in well under 30 seconds excluding the time the user takes to approve via Passkey.
- **Idempotency.** If any part of the flow fails or the process restarts mid-order, the system must not place a duplicate order. Track order state explicitly rather than assuming a request only ever runs once.

Part E — Scope boundary

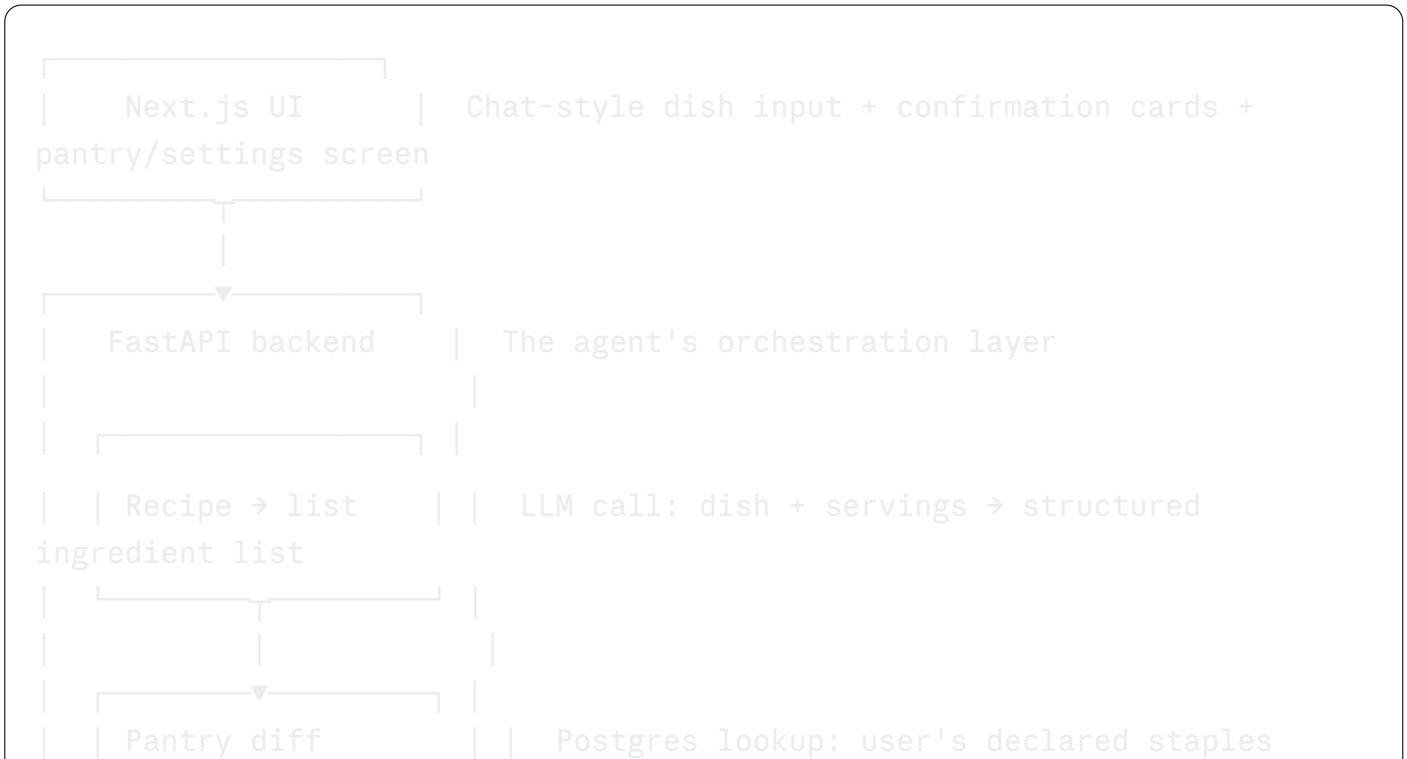
In scope

Zepto and Swiggy Instamart only, as merchant platforms. Single-recipe ordering as the primary flow. Manually maintained pantry staples list. Web UI. Cross-platform price/availability comparison and cart-splitting logic. Real Prava mandate, Passkey, and checkout integration.

Explicitly out of scope for this build

- The general Prava shopping skill (`prava-shopping`) — deliberately excluded from this build; see the earlier discussion for what it's for. It remains a possible future extension for sourcing ingredients neither platform stocks, but it is not part of the 48-hour scope.
- Any merchant beyond Zepto and Swiggy Instamart (Blinkit, BigBasket, etc. — no public MCP available for these, so they are not feasible in this timeframe regardless of interest)
- Full nutrition or macro tracking
- Auto-learned pantry detection from order history — the staples list is manually configured for this build
- A native mobile app — web only
- Handling more than one proposed substitute per out-of-stock ingredient

Part F — System architecture



Stack rationale:

- **Next.js** (Vercel-hosted) — frontend, chat UI, confirmation cards
- **FastAPI** (GCP Cloud Run) — the actual agent orchestration logic
- **Postgres** (Supabase, for fast setup) — pantry, mandates, order state
- **Zepto skill + Swiggy Instamart skill** — pre-built Prava merchant skills, no checkout code written from scratch
- **Prava SDK** — spend mandate creation, Passkey trigger, one-time payment credential issuance

---

Part G — Data model

```
User
- id

- name
- prava_account_id

PantryStaple
- id
- user_id
```

```
- ingredient_name
- always_have (bool)

SpendMandate
- id
- user_id
- platform_scope        (zepto | swiggy_instamart | both)
- cap_amount
- auto_approve_threshold
- valid_until

Order
- id
- user_id
- source_recipe           (text, e.g. "butter chicken, serves 4")
- ingredients_requested    (jsonb)
- ingredients_skipped      (jsonb — pantry matches)
- platform_chosen
- cart_items               (jsonb)
- total_amount
- status                   (pending_approval | approved | paid | failed)
- prava_session_id
- created_at
```

Part H — Agent skill setup: installing Zepto and Swiggy for Google Antigravity

The build agent for this project is Google Antigravity, a Gemini-based agentic IDE. Antigravity reads the same open `SKILL.md` format used by Claude Code, Codex, and other agents, so the Zepto and Swiggy Instamart checkout skills from `Prava-Payments/prava-skills` are installed directly rather than reimplemented.

H.1 Prerequisites

- Node.js installed (for `npx`)
- Google Antigravity installed and signed in
- This project's repo initialized locally (skills install relative to the project root for project scope)

H.2 How Antigravity discovers skills

Antigravity scans a `skills/` directory at session start and shows the agent a lightweight menu of skill names and descriptions — full instructions only load when a task actually matches one. Two directory conventions exist:

- `.agents/skills/` — current default, project scope

- `.agents/skills/` — current default, project scope
- `.agent/skills/` — legacy path, still supported

Confirm the current convention against Antigravity's own docs right before install day, since this has shifted before.

H.3 Install commands

Install the Swiggy Instamart checkout skill, scoped to Antigravity, project-level only:

```bash
npx skills add https://github.com/Prava-Payments/prava-skills \
  --skill swiggy-prava-skill \
  --agent antigravity \
  --full-depth \
  --yes
```

Install the Zepto checkout skill, scoped to Antigravity, project-level only:

```bash
npx skills add https://github.com/Prava-Payments/prava-skills \
  --skill zepto-prava-skill \
  --agent antigravity \
  --full-depth \
  --yes
```

`--full-depth` keeps sibling skills (`prava-pay`, `prava-sdk-integration`, `prava-shopping`) out of the picture — this project only needs these two, and a smaller skill menu makes Antigravity's discovery step cleaner.

To make both skills available across every project on the build machine instead of just this repo, add `--global`:

```bash
npx skills add https://github.com/Prava-Payments/prava-skills --skill swiggy-
npx skills add https://github.com/Prava-Payments/prava-skills --skill zepto-
```

If installing from a sandboxed or non-login agent shell where `npx` isn't reliably on `PATH`, use the PATH-resolving form instead:

```bash
NPX="$(command -v npx || find "$HOME/.nvm/versions/node" "$HOME/.npm-global"
  && "$NPX" --yes skills add https://github.com/Prava-Payments/prava-skills
```

(Repeat with `--skill zepto-prava-skill` for the second one.)

H.4 Verifying the install

```bash
# List every skill currently registered for Antigravity
npx skills list -a antigravity

# Confirm the files landed where expected (project scope, current default pa
ls .agents/skills/swiggy-prava-skill/SKILL.md
ls .agents/skills/zepto-prava-skill/SKILL.md
```

If Antigravity was already running, start a new session after installing — skills are enumerated at session start, not mid-session.

H.5 Keeping skills up to date

Prava skills version independently and will tell you when they're behind (this shows up as a `Skill update required (minimum: X)` message during use). Update either skill directly:

```bash
npx skills update swiggy-prava-skill -g
npx skills update zepto-prava-skill -g
```

If the update notice repeats after updating, restart the Antigravity session so it reloads the skill.

H.6 Registering the underlying MCP servers

The checkout skills orchestrate cart and checkout actions via each merchant's MCP server — the skill itself expects that MCP connection to already exist, it doesn't install the MCP server for you. Add both to the project's MCP config (Antigravity reads the same `mcp.json`-style config used by Claude Code/Codex):

```json
{
  "mcpServers": {
    "zepto": {
      "command": "npx",
      "args": ["--yes", "mcp-remote", "https://mcp.zepto.co.in/mcp"]
    },
    "swiggy": {
      "command": "npx",
      "args": ["--yes", "mcp-remote", "https://mcp.swiggy.com/mcp"]
```

```
      args": [ "-y", "mcp-remote", "https://mcp.swiggy.com/mcp" ]
    }
  }
}
```

Confirm the exact Swiggy MCP endpoint against `prava-merchants-checkout/swiggy-prava-skill/references/setup.md` in the repo before the hackathon rather than trusting the value above — merchant MCP URLs are the kind of detail most likely to have changed.

Each skill's own `references/setup.md` walks through the merchant-side auth step (Swiggy MCP access vs. Zepto MCP OAuth/mobile OTP) — read both before build day, since auth friction here is the most likely source of a demo-day surprise.

H.7 Division of responsibility (for clarity while building)

- **Skill (`SKILL.md`)** — tells the Antigravity agent *how* to use the merchant MCP to build a cart and *how* to hand off to Prava for payment. This is workflow knowledge, not code you write yourself.
- **MCP server** — the live connection that actually performs cart/search operations against Zepto or Swiggy.
- **Prava SDK/API** — integrated separately in the FastAPI backend (Part F) for mandate creation, Passkey approval, and payment token issuance. This is application code you write, not part of the skill install.

H.8 Debugging note

For Zepto checkout issues in sandboxed agent hosts, the skill ships a read-only diagnostic step before you reinstall or reauthenticate anything — it checks whether the host can find `npx`, find `prava`, read `prava status`, see the MCP config, and reach Zepto MCP tools through the fallback runner. Run that diagnostic first if checkout stops working mid-build rather than immediately tearing down the setup.

Part I — Prava integration flow (what the FastAPI backend actually calls)

1. **Mandate creation** — when the user first sets up CookCart, they create a Prava spend mandate scoped to Zepto and Swiggy Instamart, with a cap amount and an auto-approve threshold (see the `SpendMandate` model in Part G).
2. **Quote/cart build** — the cart optimizer (Part F) uses the Zepto and Swiggy skills to check availability and price per ingredient, and settles on a final cart.
3. **Confirmation card** — if the cart total is above the auto-approve threshold, the UI shows the user exactly what's about to be bought and why, and waits for explicit approval.
4. **Passkey approval** — the user approves via Touch ID/Face ID on their device. This is Prava's biometric step, not something CookCart implements itself.

is Prava's biometric step, not something CookCart implements itself.

5. **One-time credential issuance** — Prava exchanges the approved permission for a one-time, merchant-scoped payment credential (network token + dynamic CVV). The agent never sees or stores a real card number at any point.
6. **Checkout execution** — the agent uses that one-time credential to complete checkout on the chosen platform via the relevant skill.
7. **Result read-back** — checkout status is read from the payment itself (not just the network response), so a "Paid" result shown to the user is authoritative. Failed or expired quotes require a fresh quote rather than a blind retry, to avoid any risk of double-charging.

Part J — 48-hour build plan

This is a suggested hour allocation, not a rigid schedule — adjust as you go, but use it to catch scope creep early.

Hours 0–4: Setup

- Install Zepto and Swiggy skills for Antigravity (Part H)
- Register both MCP servers, run each skill's auth flow, confirm both work with a manual test cart
- Set up Next.js + FastAPI + Postgres skeleton, deploy a hello-world version to Vercel/Cloud Run so deployment isn't a last-day surprise
- Create a Prava sandbox account, link it, confirm a manual `prava shop`-style test checkout works end to end before writing any app logic

Hours 4–14: Core decision layer

- Build the recipe → ingredient list LLM call, test against the butter chicken example (Part C) until output is reliably correct
- Build the pantry staples data model and diff logic
- Build the SKU/price resolver against both skills for a single ingredient, then generalize

Hours 14–24: Cart optimization + Prava wiring

- Build the pack-size rounding logic (Part C, Step 3) with visible reasoning output
- Build the cross-platform cart comparison and split-vs-single-platform rule (Part C, Step 4)
- Wire the Prava mandate check, Passkey trigger, and checkout call (Part I)
- First full end-to-end run of the butter chicken example, real checkout, real money in sandbox

Hours 24–34: UI + reliability

- Build the confirmation card UI showing full reasoning transparency (skipped items, platform choice, price comparison)
- Add idempotency handling around order state
- Add basic error handling for the failure modes in the troubleshooting reference (expired quotes, out-of-stock items, declined payments)

Hours 34–42: Stretch features, time permitting

- Cart-splitting across platforms (F10)
- Simple substitute proposal on stockout (F12)
- Multi-dish weekly plan with ingredient deduplication (F11)

Hours 42–48: Demo prep

- Rehearse the butter chicken run multiple times end to end
- Pre-configure the auto-approve threshold so the primary demo run doesn't stall on a Passkey prompt; plan one separate live run that does show the Passkey step, so judges see it happen
- Prepare a fallback recorded run in case of live network/MCP flakiness
- Prepare the pitch: problem framing (Part A), the live demo, and the "beyond the hackathon" vision (Part K)

Part K — What happens after the hackathon

- Move from single-recipe requests to **recurring weekly meal plans** — the natural habit loop for a real product
- **Auto-detect pantry depletion** from order history instead of a manually maintained staples list
- Offer the decision layer (recipe → ingredients → cross-platform cart) as an **embeddable feature** for existing meal-planning or fitness apps via the Prava SDK pattern — a plausible B2B angle
- Extend sourcing to `prava-shopping` for ingredients neither Zepto nor Swiggy Instamart stocks, once the core two-platform flow is solid

---

Part L — Risks

Risk	Mitigation
Zepto/Swiggy MCP auth or rate	Pre-authenticate well in advance, test the full flow multiple

limits fail during the live demo	times, keep a recorded fallback run ready
LLM misjudges ingredient quantities on an unfamiliar recipe	Rehearse the butter chicken example specifically; if taking audience recipe suggestions, treat it as a deliberate "riskier" bonus moment, not the core demo
Pack-size rounding produces an odd-looking result (e.g. buying 1kg for a 600g need)	Always show the reasoning in the confirmation card — visible reasoning turns an imperfect decision into a trust feature rather than a bug
Passkey approval adds friction/delay mid-demo	Pre-set the auto-approve threshold above the primary demo's total; run one separate above-threshold order specifically to show the Passkey step
Running out of time for cross-platform splitting or substitution	These are explicitly marked Should/Could in Part D — cut them first if time runs short, never cut the core single-platform flow

Part M — Success criteria for the submission

- ▢ Live demo: user types a dish + servings, agent completes an actual paid order on Zepto or Swiggy Instamart, within the spend cap
- ▢ Pantry diff visibly changes the ingredient list — not cosmetic, it actually removes items
- ▢ Cross-platform comparison is real and explainable, not hardcoded — judges can ask "why this platform" and get a genuine answer
- ▢ Prava mandate and Passkey approval are shown on-screen, not just claimed in the pitch
- ▢ The post-hackathon product vision (Part K) is clearly articulated during the pitch